

Prueba de Caja Blanca

“Sistema automatizado para la administración de ingresos en el conjunto Fortín del Valle”

Integrantes:

Joshep Chisaguano

Cristian Suquillo

Melanie Zuñiga

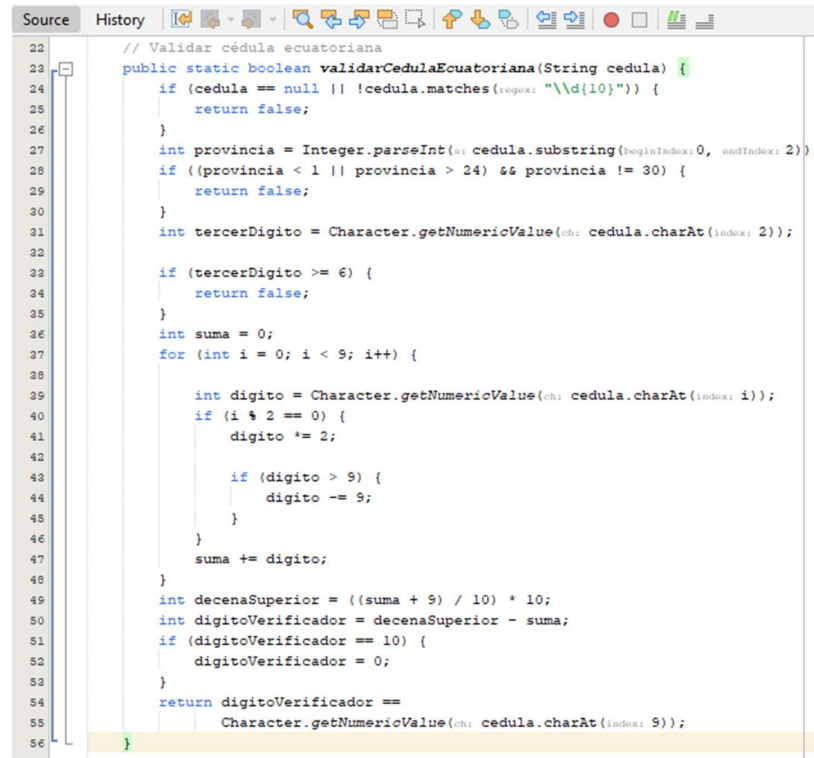
Paul Villacis

Fecha: 2026/06/19

Requisito funcional N° 3 Registro de Residentes

El sistema deberá permitir el registro de residentes, validando cada campo para un mejor manejo de la información para el administrador

1. CÓDIGO FUENTE



```
22 // Validar cédula ecuatoriana
23 public static boolean validarCedulaEcuatoriana(String cedula) {
24     if (cedula == null || !cedula.matches("\\d{10}") {
25         return false;
26     }
27     int provincia = Integer.parseInt(cedula.substring(beginIndex: 0, endIndex: 2));
28     if ((provincia < 1 || provincia > 24) && provincia != 30) {
29         return false;
30     }
31     int tercerDigito = Character.getNumericValue(cedula.charAt(index: 2));
32
33     if (tercerDigito >= 6) {
34         return false;
35     }
36     int suma = 0;
37     for (int i = 0; i < 9; i++) {
38
39         int digito = Character.getNumericValue(cedula.charAt(index: i));
40         if (i % 2 == 0) {
41             digito *= 2;
42
43             if (digito > 9) {
44                 digito -= 9;
45             }
46         }
47         suma += digito;
48     }
49     int decenaSuperior = ((suma + 9) / 10) * 10;
50     int digitoVerificador = decenaSuperior - suma;
51     if (digitoVerificador == 10) {
52         digitoVerificador = 0;
53     }
54     return digitoVerificador ==
55         Character.getNumericValue(cedula.charAt(index: 9));
56 }
```

```
public static boolean validarCedulaEcuatoriana(String cedula) {
```

```
    if (cedula == null || !cedula.matches("\\d{10}") {
        return false;
    }
```

```
    int provincia = Integer.parseInt(cedula.substring(0, 2));
```

```
    if ((provincia < 1 || provincia > 24) && provincia != 30) {
        return false;
    }
```

```
    int tercerDigito = Character.getNumericValue(cedula.charAt(2));
```

```
    if (tercerDigito >= 6) {
        return false;
    }
```

```
    int suma = 0;
```

```
    for (int i = 0; i < 9; i++) {
```

```
        int digito = Character.getNumericValue(cedula.charAt(i));
```

```

        if (i % 2 == 0) {
            digito *= 2;

            if (digito > 9) {
                digito -= 9;
            }
        }

        suma += digito;
    }

    int decenaSuperior = ((suma + 9) / 10) * 10;
    int digitoVerificador = decenaSuperior - suma;

    if (digitoVerificador == 10) {
        digitoVerificador = 0;
    }

    return digitoVerificador ==
        Character.getNumericValue(cedula.charAt(9));
}

```

// Validación completa del formulario

```

58 // Validación completa del formulario
59 public static String validarResidente(Residentes r) {
60     // Campos vacíos
61     if (campoVacio(texto: r.getNombres()))
62         return "Ingrese los nombres.";
63     if (campoVacio(texto: r.getApellidos()))
64         return "Ingrese los apellidos.";
65     if (campoVacio(texto: r.getCedula()))
66         return "Ingrese la cédula.";
67     if (campoVacio(texto: r.getTelefonoMovil()))
68         return "Ingrese el teléfono móvil.";
69     if (campoVacio(texto: r.getTelefonoConvencional()))
70         return "Ingrese el teléfono convencional.";
71     if (campoVacio(texto: r.getDireccion()))
72         return "Ingrese la dirección.";
73     if (campoVacio(texto: r.getNumeroVehiculos()))
74         return "Ingrese el número de vehículos.";
75     if (campoVacio(texto: r.getNumeroPersonas()))
76         return "Ingrese el número de personas que viven con el propietario.";
77     if (campoVacio(texto: r.getNombresResidentes()))
78         return "Ingrese los nombres de los residentes.";
79     if (campoVacio(texto: r.getApellidosResidentes()))
80         return "Ingrese los apellidos de los residentes.";
81     // Nombres y apellidos sin números
82     if (!soloLetras(texto: r.getNombres()))
83         return "Los nombres no deben contener números.";
84     if (!soloLetras(texto: r.getApellidos()))
85         return "Los apellidos no deben contener números.";
86     if (!soloLetras(texto: r.getNombresResidentes()))
87         return "Los nombres de los residentes no deben contener números.";
88     if (!soloLetras(texto: r.getApellidosResidentes()))
89         return "Los apellidos de los residentes no deben contener números.";
90     // Cédula ecuatoriana
91     if (!validarCedulaEcuatoriana(cedula: r.getCedula()))
92         return "La cédula ecuatoriana ingresada no es válida.";

```

```

93 // Teléfono móvil (10 dígitos)
94 if (!r.getTelefonoMovil().matches(regex: "\\d{10}"))
95     return "El teléfono móvil debe contener exactamente 10 números.";
96 // Teléfono convencional (7 dígitos)
97 if (!r.getTelefonoConvencional().matches(regex: "\\d{7}"))
98     return "El teléfono convencional debe contener exactamente 7 números.";
99 // Número de vehículos
100 if (!soloNumeros(texto: r.getNumeroVehiculos()))
101     return "El número de vehículos solo debe contener números.";
102 // Número de personas
103 if (!soloNumeros(texto: r.getNumeroPersonas()))
104     return "El número de personas solo debe contener números.";
105 return null; // Todo correcto
106 }
107

```

```

public static String validarResidente(Residentes r) {

```

```

    // Campos vacíos

```

```

    if (campoVacio(r.getNombres()))
        return "Ingrese los nombres.";

```

```

    if (campoVacio(r.getApellidos()))
        return "Ingrese los apellidos.";

```

```

    if (campoVacio(r.getCedula()))
        return "Ingrese la cédula.";

```

```

    if (campoVacio(r.getTelefonoMovil()))
        return "Ingrese el teléfono móvil.";

```

```

    if (campoVacio(r.getTelefonoConvencional()))
        return "Ingrese el teléfono convencional.";

```

```

    if (campoVacio(r.getDireccion()))
        return "Ingrese la dirección.";

```

```

    if (campoVacio(r.getNumeroVehiculos()))
        return "Ingrese el número de vehículos.";

```

```

    if (campoVacio(r.getNumeroPersonas()))
        return "Ingrese el número de personas que viven con el propietario.";

```

```

    if (campoVacio(r.getNombresResidentes()))
        return "Ingrese los nombres de los residentes.";

```

```

    if (campoVacio(r.getApellidosResidentes()))
        return "Ingrese los apellidos de los residentes.";

```

```

    // Nombres y apellidos sin números

```

```

    if (!soloLetras(r.getNombres()))
        return "Los nombres no deben contener números.";

```

```

    if (!soloLetras(r.getApellidos()))

```

```

        return "Los apellidos no deben contener números.";

        if (!soloLetras(r.getNombresResidentes()))
            return "Los nombres de los residentes no deben contener números.";

        if (!soloLetras(r.getApellidosResidentes()))
            return "Los apellidos de los residentes no deben contener números.";

        // Cédula ecuatoriana
        if (!validarCedulaEcuatoriana(r.getCedula()))
            return "La cédula ecuatoriana ingresada no es válida.";

        // Teléfono móvil (10 dígitos)
        if (!r.getTelefonoMovil().matches("\\d{10}"))
            return "El teléfono móvil debe contener exactamente 10 números.";

        // Teléfono convencional (7 dígitos)
        if (!r.getTelefonoConvencional().matches("\\d{7}"))
            return "El teléfono convencional debe contener exactamente 7 números.";

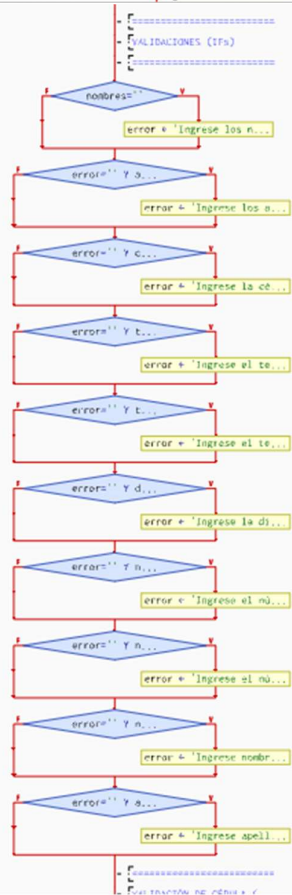
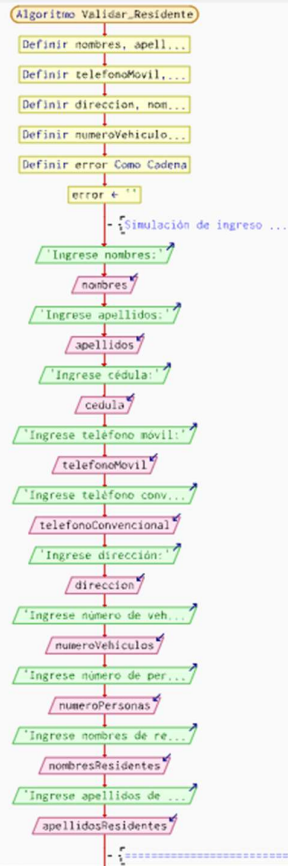
        // Número de vehículos
        if (!soloNumeros(r.getNumeroVehiculos()))
            return "El número de vehículos solo debe contener números.";

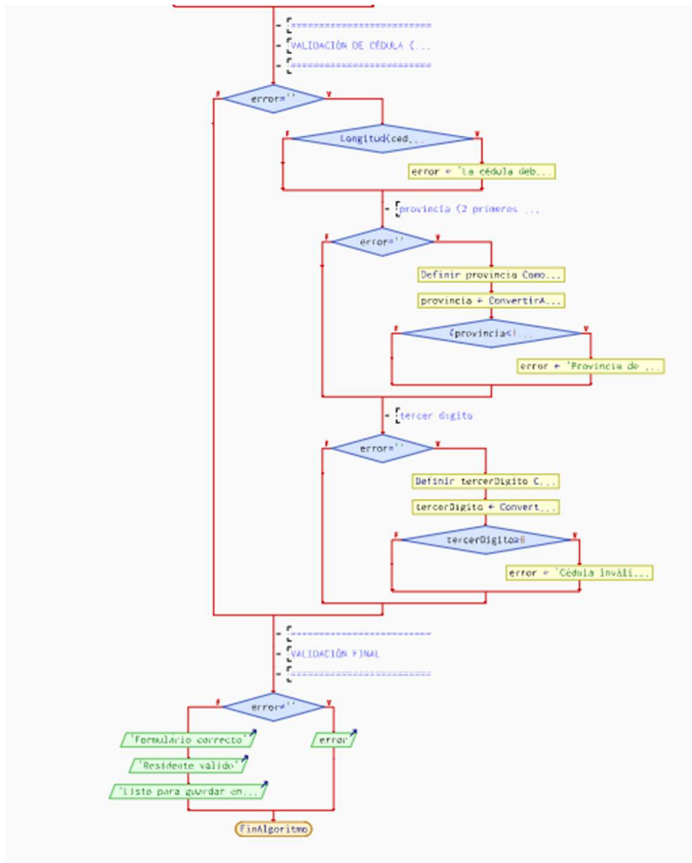
        // Número de personas
        if (!soloNumeros(r.getNumeroPersonas()))
            return "El número de personas solo debe contener números.";

        return null; // Todo correcto
    }

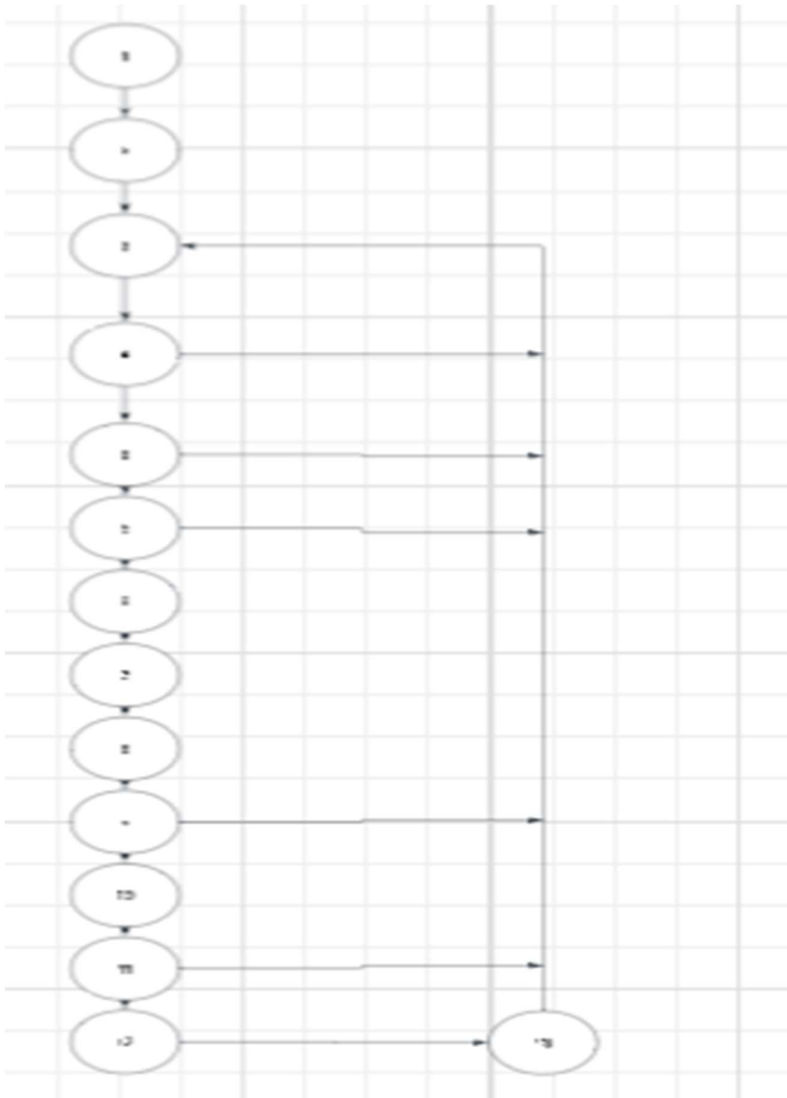
```

2. DIAGRAMA DE FLUJO (DF)





3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

RUTAS

Ruta	Secuencia	Descripción
R1	1→2→3→4→6→8→10→12→14→16→18→20→22→23→25→26→29→31→33→34	Todo correcto, formulario válido
R2	1→2→3→4→6→8→10→12→14→16→18→20→22→31→32→34	error≠"", salta bloque cédula → muestra error

R3	1→2→4→6→8→10→12→14→16→18→20→22→23→25→26→29→31→32→34	nombres vacío → error
R4	1→2→3→4→5→6→8→10→12→14→16→18→20→22→23→25→26→29→31→32→34	apellidos vacío → error
R5	1→2→3→4→6→7→8→10→12→14→16→18→20→22→23→25→26→29→31→32→34	cédula vacía → error
R6	1→2→3→4→6→8→9→10→12→14→16→18→20→22→23→25→26→29→31→32→34	teléfono móvil vacío → error
R7	1→2→3→4→6→8→10→11→12→14→16→18→20→22→23→25→26→29→31→32→34	teléfono convencional vacío → error
R8	1→2→3→4→6→8→10→12→13→14→16→18→20→22→23→25→26→29→31→32→34	dirección vacía → error
R9	1→2→3→4→6→8→10→12→14→15→16→18→20→22→23→25→26→29→31→32→34	número vehículos vacío → error
R10	1→2→3→4→6→8→10→12→14→16→17→18→20→22→23→25→26→29→31→32→34	número personas vacío → error
R11	1→2→3→4→6→8→10→12→14→16→18→19→20→22→23→25→26→29→31→32→34	nombres residentes vacío → error
R12	1→2→3→4→6→8→10→12→14→16→18→20→21→22→23→25→26→29→31→32→34	apellidos residentes vacío → error
R13	1→2→3→4→6→8→10→12→14→16→18→20→22→23→24→25→26→29→31→32→34	longitud cédula ≠ 10 → error
R14	1→2→3→4→6→8→10→12→14→16→18→20→22→23→25→28→31→32→34	error=""" después longitud, salta provincia → error tercer dígito
R15	1→2→3→4→6→8→10→12→14→16→18→20→22→23→25→26→27→28→29→31→32→34	provincia inválida → error
R16	1→2→3→4→6→8→10→12→14→16→18→20→22→23→25→26→28→29→31→32→34	provincia válida, salta a tercer dígito

R17	1→2→3→4→6→8→10→12→14→16→18→20→22→23→25→26→29→30→31→32→34	tercer dígito >= 6 → error
R18	1→2→3→4→6→8→10→12→14→16→18→20→22→28→31→33→34	error="" salta provincia y tercer dígito → formulario correcto

5. COMPLEJIDAD CICLOMÁTICA eliminación de contratos

Se puede calcular de las siguientes formas:

DATOS:

P: Número de nodos predicado: 17

A: Número de aristas: 37

N: Número de nodos: 21

V(G) Nodos predicados (IF-THEN-ELSE) +1

$V(G) = P+1$

$V(G) = 17+1 = 18$

$V(G) = A-N + 2$

$V(G) = 37-21+2$

$V(G)=18$

Resultado

La complejidad ciclomática del requisito Registro de Residentes es 18, lo que indica que existen dieciocho caminos básicos independientes.

DONDE

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos